

API Reference

Jupiter System for PC API Reference



Document Version 2006.1.17
Jupiter Build 69

Legal Notices

The Jupiter™ Development System is copyright 1998-2006 by Touchdown Entertainment, Inc., Kirkland, Washington, U.S.A. All rights reserved.

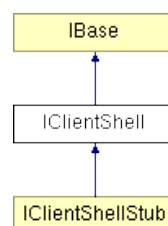
ImageMagick is copyright 2000 by ImageMagick Studio, a non-profit organization dedicated to making software imaging solutions freely available.

The FreeType Project is copyright 1996-2000 by David Turner, Robert Wilhelm, and Werner Lemberg. All rights reserved except as specified in the *Credits* appendix.

API Reference

IClientShell Class Reference

Inheritance diagram for IClientShell:



Public Types

```
enum { IClientShell\_VERSION = 4 }
```

Public Member Functions

```

virtual void PreUpdate ()=0
virtual void PostUpdate ()=0
virtual void Update ()=0
virtual LTRESULT OnObjectRotate (HLOCALOBJ hObj, bool bTeleport, LTRotation *pNewRot)=0
virtual LTRESULT OnEngineInitialized (RMode *pMode, LTGUID *pAppGuid)=0
    virtual void OnEngineTerm ()=0
    virtual void OnEvent (uint32 dwEventID, uint32 dwParam)=0
virtual LTRESULT OnTouchNotify (HOBJECT hMain, CollisionInfo *pInfo, float forceMag)=0
    virtual void OnPlaySound (PlaySoundInfo *pPlaySoundInfo)=0
virtual LTRESULT ProcessPacket (ILTMessage\_Read *pMsg, uint8 senderAddr[4], uint16 senderPort)=0
    virtual void OnLockRenderer ()=0
    virtual void OnUnLockRenderer ()=0
    virtual void OnConsolePrint (CConsolePrintData *pData)=0
    virtual void OnKeyUp (int key)=0
    virtual void OnKeyDown (int key, int rep)=0
    virtual void OnCommandOff (int command)=0
    virtual void OnCommandOn (int command)=0
    virtual void OnModelKey (HLOCALOBJ hObj, ArgList *pArgs)=0
    virtual void OnEnterWorld ()=0
    virtual void OnExitWorld ()=0
    virtual void OnMessage (ILTMessage\_Read *pMessage)=0
    virtual void PreLoadWorld (const char *pWorldName)=0
virtual LTRESULT OnObjectMove (HLOCALOBJ hObj, bool bTeleport, LTVector *pNewPos)=0
    virtual void SpecialEffectNotify (HLOCALOBJ hObj, ILTMessage\_Read *pMessage)=0
    virtual void OnObjectRemove (HLOCALOBJ hObj)=0
    virtual void OnVertexShaderSetConstants (LTVertexShader *pVertexShader, unsigned iPass, CRenderStyle
        *pRenderStyle, HOBJECT hObj, const LTShaderDeviceState &ShaderDeviceState)
    virtual void OnPixelShaderSetConstants (LTPixelShader *pPixelShader, unsigned iPass, CRenderStyle
        *pRenderStyle, HOBJECT hObj, const LTShaderDeviceState &ShaderDeviceState)
    virtual void OnEffectShaderSetParams (LTEffectShader *pEffect, CRenderStyle *pRenderStyle, HOBJECT hObj,
        const LTShaderDeviceState &ShaderDeviceState)

```

Detailed Description

IClientShell interface. This is full of callback functions that Lithtech will call to notify you of things.

Member Enumeration Documentation

anonymous enum

Enumeration values:

[_IClientShell_VERSION_](#)

Member Function Documentation

virtual void IClientShell::OnCommandOff(int *command*) [pure virtual]

Parameters:

command ID of the command that was turned off.

Called whenever a "command" is turned off (i.e. usually when a key is released). Input is processed during each update loop, before [PreUpdate\(\)](#) is called.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

virtual void IClientShell::OnCommandOn(int *command*) [pure virtual]

Parameters:

command ID of the command that was turned on.

Called whenever a "command" is turned on (i.e. usually when a key is pressed). Input is processed during each update loop, before [PreUpdate\(\)](#) is called.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

```
virtual void IClientShell::OnConsolePrint( CConsolePrintData * pData ) [pure virtual]
```

Parameters:

pData Pointer to a [CConsolePrintData](#) structure containing information about the string to be printed.

Returns:

???

Called when the engine wishes to send output to the console. Any changes to the *pData* structure are reflected in the output, so setting *pData*->*m_pMessage* to NULL will prevent the engine from printing a message to the console, etc.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

```
virtual void IClientShell::OnEffectShaderSetParams( LTEffectShader *      pEffect,
                                                    CRenderStyle *      pRenderStyle,
                                                    HOBJECT             hObj,
                                                    const LTShaderDeviceState & ShaderDeviceState
                                                    ) [inline, virtual]
```

Parameters:

pEffect Pointer to the current effect shader.
pRenderStyle Pointer to the current RenderStyle. (Only valid for models)
hObj Object that is about to be rendered. (Only valid for models)
ShaderDeviceState Object that allows access to device transforms, lights, and the camera.

This function is called just before a effect shader's constants are sent to the video card.

Used for: Rendering.

```
virtual LTRESULT IClientShell::OnEngineInitialized( RMode * pMode,
                                                LTGUID * pAppGuid
                                                ) [pure virtual]
```

Parameters:

pMode Render mode (initially filled in by engine with values defined in autoexec.cfg).
pAppGuid (output) Application Globally Unique Identifier. Be sure to initialize this before returning. This controls what netgames you can query and which ones you can connect to.

Returns:

LT_OK - Everything went fine.

LT_ERROR - A fatal error occurred.

Called after the engine has been fully initialized and is ready to go. The engine doesn't initialize the renderer itself, but passes in the default [RMode](#) it would use to initialize the renderer (from the "RenderDLL", "ScreenWidth", and "ScreenHeight" console variables). You must call SetRenderMode to initialize the renderer in here. Initialize *pAppGuid* before returning. If this function returns something other than LT_OK, the engine will shut down immediately. Typically, you would also initialize any client-side lists, managers, etc, and create a camera object. Additionally, this function can process command-line arguments, and set the initial game mode (go to a menu, run a world, etc).

Note: You MUST override this function in your client shell, or else LT_ERROR will be returned and the game will not run.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

virtual void IClientShell::OnEngineTerm() [pure virtual]

Called just before the engine unloads the clientshell and shuts down.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

virtual void IClientShell::OnEnterWorld() [pure virtual]

Called when the client enters a world (after the world has been loaded and all objects have been loaded, etc).

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

virtual void IClientShell::OnEvent([uint32](#) *dwEventID*,
[uint32](#) *dwParam*
) [pure virtual]

Parameters:

dwEventID ID of event (one of the LTEVENT_ defines)

dwParam ???

Called when a particular engine event happens (e.g. the game lost focus due to the player pressing ALT-TAB, or the renderer was initialized, etc).

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

virtual void IClientShell::OnExitWorld() [pure virtual]

Called when the client exits a world (after disconnecting/unloading the server and all objects have been deleted). You lose *all* server objects when you exit the world, so don't reference them after exiting the world!

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

virtual void IClientShell::OnKeyDown(int *key*,
int *rep*
) [pure virtual]

Parameters:

key The keycode for the key that was pressed.

rep number of repetitions that have occurred since the last update.

Called each update loop if a key is pressed (or held). Try to use [OnCommandOn\(\)](#) whenever possible because key up/down notification is not portable. Input is processed during each update loop, before [PreUpdate\(\)](#) is called.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

virtual void IClientShell::OnKeyUp(int *key*) [pure virtual]

Parameters:

key The keycode for the key that was released.

Called if a key is released. Try to use [OnCommandOff\(\)](#) whenever possible because key up/down notification is not portable. Input is processed during each update loop, before [PreUpdate\(\)](#) is called.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

virtual void IClientShell::OnLockRenderer() [pure virtual]

The game should pause any secondary rendering threads when this function is called.

Implemented in [IClientShellStub](#).

virtual void IClientShell::OnMessage([ILTMessage_Read](#) * *pMessage*) [pure virtual]

Parameters:

messageID ID of message. This is a custom ID, not an engine ID.

pMessage The message.

Called whenever the client receives a message from the server. These messages are those that are defined and sent via game code.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

virtual void IClientShell::OnModelKey([HLOCALOBJ](#) *hObj*,
[ArgList](#) * *pArgs*
) [pure virtual]

Parameters:

hObj The model that is currently hitting the frame key.

pArgs The frame string, broken up into an array of tokens.

Called when a model object with client flag CF_NOTIFYMODELKEYS hits a frame key.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

virtual [LTRESULT](#) IClientShell::OnObjectMove([HLOCALOBJ](#) *hObj*,
 bool *bTeleport*,
[LTVector](#) * *pNewPos*
) [pure virtual]

Parameters:

hObj The object that is about to move.

bTeleport Teleport status. True = object is teleporting. False = object is moving normally, and thus physics are applied.

pNewPos The destination position. This CAN be intercepted and modified in this function.

Returns:

???

Called just before the engine moves an object. *pNewPos* can be modified at this time.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

virtual void IClientShell::OnObjectRemove([HLOCALOBJ](#) *hObj*) [pure virtual]

Parameters:

hObj The object that is about to be removed.

Called when an object with client flag CF_NOTIFYONREMOVE is removed.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

virtual [LTRESULT](#) IClientShell::OnObjectRotate([HLOCALOBJ](#) *hObj*,
 bool *bTeleport*,

```

    LTRotation * pNewRot
) [pure virtual]

```

Parameters:

hObj The object that is about to rotate.

bTeleport Teleport status. True = object is teleporting. False = object is moving normally, and thus physics are applied.

pNewRot The destination rotation. This CAN be intercepted and modified in this function.

Returns:

???

Called just before the engine rotates an object. pNewRot can be modified at this time.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

```

virtual void IClientShell::OnPixelShaderSetConstants( LTPixelShader *      pPixelShader,
                                                    unsigned          iPass,
                                                    CRenderStyle *      pRenderStyle,
                                                    HOBJECT             hObj,
                                                    const LTShaderDeviceState & ShaderDeviceState
                                                    ) [inline, virtual]

```

Parameters:

pPixelShader Pointer to the current pixel shader.

iPass Current render pass number.

pRenderStyle Pointer to the current RenderStyle.

hObj Object that is about to be rendered.

ShaderDeviceState Object that allows access to device transforms, lights, and the camera.

This function is called just before a pixel shader's constants are sent to the video card.

Used for: Models.

```

virtual void IClientShell::OnPlaySound( PlaySoundInfo * pPlaySoundInfo ) [pure virtual]

```

Parameters:

pPlaySoundInfo Description of sound (provided by engine).

Called just after a sound is played by the client or server.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

```

virtual LTRESULT IClientShell::OnTouchNotify( HOBJECT      hMain,
                                           CollisionInfo * pInfo,
                                           float          forceMag
                                           ) [pure virtual]

```

Parameters:

hMain The object involved in this collision.

pInfo Collision info (provided by engine).

forceMag Magnitude of collision (provided by engine).

Returns:

???

Called when an object (with FLAG_TOUCH_NOTIFY) moved on the client collides with another object or the world.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

```
virtual void IClientShell::OnUnLockRenderer( ) [pure virtual]
```

The game should resume any secondary rendering threads when this function is called.

Implemented in [IClientShellStub](#).

```
virtual void IClientShell::OnVertexShaderSetConstants( LTVertexShader *      pVertexShader,
                                                       unsigned                iPass,
                                                       CRenderStyle *      pRenderStyle,
                                                       HOBJECT              hObj,
                                                       const LTShaderDeviceState & ShaderDeviceState
                                                       ) [inline, virtual]
```

Parameters:

pVertexShader Pointer to the current vertex shader.
iPass Current render pass number.
pRenderStyle Pointer to the current RenderStyle.
hObj Object that is about to be rendered.
ShaderDeviceState Object that allows access to device transforms, lights, and the camera.

This function is called just before a vertex shader's constants are sent to the video card.

Used for: Models.

```
virtual void IClientShell::PostUpdate( ) [pure virtual]
```

Called at the end of every update cycle -- after [Update\(\)](#), *and* after all client-side objects have been processed by the engine.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

```
virtual void IClientShell::PreLoadWorld( const char * pWorldName ) [pure virtual]
```

Parameters:

pWorldName Name of the world about to be loaded.

Called right before a world is loaded. Typically this function displays a "loading" screen.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

```
virtual void IClientShell::PreUpdate( ) [pure virtual]
```

Called at the beginning of every update cycle -- before [Update\(\)](#).

Note: Technically, [Update\(\)](#) is called immediately after [PreUpdate\(\)](#). Therefore, the only benefit to having code in [PreUpdate\(\)](#) is purely organizational.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

```
virtual LTRESULT IClientShell::ProcessPacket( LTMessage\_Read * pMsg,
                                             uint8             senderAddr[4],
                                             uint16            senderPort
                                             ) [pure virtual]
```

Parameters:

pData The packet data.
dataLen Length of the packet data.
senderAddr IP address of sender.
senderPort Port of sender.

Returns:

The engine ignores the callback return value.

Called when the client gets a network packet that it cannot understand. Typically, this is used in conjunction with apps like GameSpy, etc.

Used for: ServerShell Callbacks.

Implemented in [IClientShellStub](#).

```
virtual void IClientShell::SpecialEffectNotify( HLOCALOBJ      hObj,
                                              ILTMessage\_Read *pMessage
                                              ) [pure virtual]
```

Parameters:

hObj The object associated with the special effect. If NULL, then this is an "instant" special effect.
pMessage Special effect message.

Called when the server tells the client about a special effect on an object. If hObj is NULL, then this is an "instant" special effect (ie. it's not associated with an object, but instead, with a location). These messages are those that are defined by game code, and are usually (but not necessarily) sent via game code.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

```
virtual void IClientShell::Update( ) [pure virtual]
```

Called during every update cycle. This is the function that usually updates all client-side systems, objects, etc.

Used for: ClientShell Callbacks.

Implemented in [IClientShellStub](#).

The documentation for this class was generated from the following file:

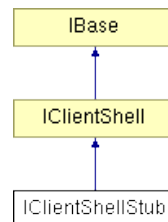
- [iclientshell.h](#)
-

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

IClientShellStub Class Reference

Inheritance diagram for IClientShellStub:



Public Member Functions

virtual void [OnCommandOn](#) (int command)
 virtual void [OnCommandOff](#) (int command)
 virtual void [OnKeyDown](#) (int key, int rep)


```

virtual void OnKeyUp (int key)
virtual void OnMessage (ILTMessage\_Read *pMessage)
virtual void OnModelKey (HLOCALOBJ hObj, ArgList *pArgs)
virtual void PreLoadWorld (const char *pWorldName)
virtual void OnEnterWorld ()
virtual void OnExitWorld ()
virtual void SpecialEffectNotify (HLOCALOBJ hObj, ILTMessage\_Read *pMessage)
virtual void OnObjectRemove (HLOCALOBJ hObj)
virtual void PreUpdate ()
virtual void Update ()
virtual void PostUpdate ()
virtual LTRESULT OnObjectMove (HLOCALOBJ hObj, bool bTeleport, LTVector *pNewPos)
virtual LTRESULT OnObjectRotate (HLOCALOBJ hObj, bool bTeleport, LTRotation *pNewRot)
virtual LTRESULT OnEngineInitialized (RMode *pMode, LTGUID *pAppGuid)
    virtual void OnEngineTerm ()
    virtual void OnEvent (uint32 dwEventID, uint32 dwParam)
virtual LTRESULT OnTouchNotify (HOBJECT hMain, CollisionInfo *pInfo, float forceMag)
    virtual void SRand ()
    virtual void OnPlaySound (PlaySoundInfo *pPlaySoundInfo)
    virtual void OnConsolePrint (CConsolePrintData *pData)
virtual LTRESULT ProcessPacket (ILTMessage\_Read *pMsg, uint8 senderAddr[4], uint16 senderPort)
    virtual void OnLockRenderer ()
    virtual void OnUnLockRenderer ()

```

Detailed Description

IClientShellStub class. This class gives empty bodies to all [IClientShell](#) functions. Derive your [IClientShell](#) class from IClientShellStub and you only need to declare functions that have non-trivial bodies.

Member Function Documentation

```
virtual void IClientShellStub::OnCommandOff( int command ) [inline, virtual]
```

Parameters:

command ID of the command that was turned off.

Called whenever a "command" is turned off (i.e. usually when a key is released). Input is processed during each update loop, before [PreUpdate\(\)](#) is called.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnCommandOn( int command ) [inline, virtual]
```

Parameters:

command ID of the command that was turned on.

Called whenever a "command" is turned on (i.e. usually when a key is pressed). Input is processed during each update loop, before [PreUpdate\(\)](#) is called.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnConsolePrint( CConsolePrintData * pData ) [inline, virtual]
```

Parameters:

pData Pointer to a [CConsolePrintData](#) structure containing information about the string to be printed.

Returns:

???

Called when the engine wishes to send output to the console. Any changes to the pData structure are reflected in the output, so setting pData->m_pMessage to NULL will prevent the engine from printing a message to the console, etc.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual LTRESULT IClientShellStub::OnEngineInitialized( RMode * pMode,
                                                    LTGUID * pAppGuid
                                                    ) [inline, virtual]
```

Parameters:

pMode Render mode (initially filled in by engine with values defined in autoexec.cfg).
pAppGuid (output) Application Globally Unique Identifier. Be sure to initialize this before returning. This controls what netgames you can query and which ones you can connect to.

Returns:

LT_OK - Everything went fine.

LT_ERROR - A fatal error occurred.

Called after the engine has been fully initialized and is ready to go. The engine doesn't initialize the renderer itself, but passes in the default [RMode](#) it would use to initialize the renderer (from the "RenderDLL", "ScreenWidth", and "ScreenHeight" console variables). You must call SetRenderMode to initialize the renderer in here. Initialize pAppGuid before returning. If this function returns something other than LT_OK, the engine will shut down immediately. Typically, you would also initialize any client-side lists, managers, etc, and create a camera object. Additionally, this function can process command-line arguments, and set the initial game mode (go to a menu, run a world, etc).

Note: You MUST override this function in your client shell, or else LT_ERROR will be returned and the game will not run.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnEngineTerm( ) [inline, virtual]
```

Called just before the engine unloads the clientshell and shuts down.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnEnterWorld( ) [inline, virtual]
```

Called when the client enters a world (after the world has been loaded and all objects have been loaded, etc).

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnEvent( uint32 dwEventID,
                                       uint32 dwParam
                                       ) [inline, virtual]
```

Parameters:

dwEventID ID of event (one of the LTEVENT_ defines)
dwParam ???

Called when a particular engine event happens (e.g. the game lost focus due to the player pressing ALT-TAB, or the renderer was initialized, etc).

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnExitWorld( ) [inline, virtual]
```

Called when the client exits a world (after disconnecting/unloading the server and all objects have been deleted). You lose *all* server objects when you exit the world, so don't reference them after exiting the world!

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnKeyDown( int      key,
                                         int      rep
                                         ) [inline, virtual]
```

Parameters:

key The keycode for the key that was pressed.
rep number of repetitions that have occurred since the last update.

Called each update loop if a key is pressed (or held). Try to use [OnCommandOn\(\)](#) whenever possible because key up/down notification is not portable. Input is processed during each update loop, before [PreUpdate\(\)](#) is called.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnKeyUp( int key ) [inline, virtual]
```

Parameters:

key The keycode for the key that was released.

Called if a key is released. Try to use [OnCommandOff\(\)](#) whenever possible because key up/down notification is not portable. Input is processed during each update loop, before [PreUpdate\(\)](#) is called.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnLockRenderer( ) [inline, virtual]
```

The game should pause any secondary rendering threads when this function is called.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnMessage( ILTMessage\_Read * pMessage ) [inline, virtual]
```

Parameters:

messageID ID of message. This is a custom ID, not an engine ID.
pMessage The message.

Called whenever the client receives a message from the server. These messages are those that are defined and sent via game code.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnModelKey( HLOCALOBJ hObj,
                                           ArgList *   pArgs
                                           ) [inline, virtual]
```

Parameters:

hObj The model that is currently hitting the frame key.
pArgs The frame string, broken up into an array of tokens.

Called when a model object with client flag CF_NOTIFYMODELKEYS hits a frame key.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual LTRESULT IClientShellStub::OnObjectMove( HLOCALOBJ hObj,
                                                bool          bTeleport,
                                                LTVector *    pNewPos
                                              ) [inline, virtual]
```

Parameters:

hObj The object that is about to move.
bTeleport Teleport status. True = object is teleporting. False = object is moving normally, and thus physics are applied.
pNewPos The destination position. This CAN be intercepted and modified in this function.

Returns:

???

Called just before the engine moves an object. pNewPos can be modified at this time.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnObjectRemove( HLOCALOBJ hObj ) [inline, virtual]
```

Parameters:

hObj The object that is about to be removed.

Called when an object with client flag CF_NOTIFYONREMOVE is removed.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual LTRESULT IClientShellStub::OnObjectRotate( HLOCALOBJ hObj,
                                                  bool          bTeleport,
                                                  LTRotation * pNewRot
                                                ) [inline, virtual]
```

Parameters:

hObj The object that is about to rotate.
bTeleport Teleport status. True = object is teleporting. False = object is moving normally, and thus physics are applied.
pNewRot The destination rotation. This CAN be intercepted and modified in this function.

Returns:

???

Called just before the engine rotates an object. pNewRot can be modified at this time.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnPlaySound( PlaySoundInfo * pPlaySoundInfo ) [inline, virtual]
```

Parameters:

pPlaySoundInfo Description of sound (provided by engine).

Called just after a sound is played by the client or server.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual LTRESULT IClientShellStub::OnTouchNotify( HOBJECT hMain,
```

```

        CollisionInfo * pInfo,
        float forceMag
    ) [inline, virtual]

```

Parameters:

hMain The object involved in this collision.
pInfo Collision info (provided by engine).
forceMag Magnitude of collision (provided by engine).

Returns:

???

Called when an object (with FLAG_TOUCH_NOTIFY) moved on the client collides with another object or the world.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::OnUnLockRenderer( ) [inline, virtual]
```

The game should resume any secondary rendering threads when this function is called.

Implements [IClientShell](#).

```
virtual void IClientShellStub::PostUpdate( ) [inline, virtual]
```

Called at the end of every update cycle -- after [Update\(\)](#), *and* after all client-side objects have been processed by the engine.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::PreLoadWorld( const char * pWorldName ) [inline, virtual]
```

Parameters:

pWorldName Name of the world about to be loaded.

Called right before a world is loaded. Typically this function displays a "loading" screen.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::PreUpdate( ) [inline, virtual]
```

Called at the beginning of every update cycle -- before [Update\(\)](#).

Note: Technically, [Update\(\)](#) is called immediately after [PreUpdate\(\)](#). Therefore, the only benefit to having code in [PreUpdate\(\)](#) is purely organizational.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual LTRESULT IClientShellStub::ProcessPacket( ILTMessage\_Read * pMsg,
        uint8 senderAddr[4],
        uint16 senderPort
    ) [inline, virtual]
```

Parameters:

pData The packet data.
dataLen Length of the packet data.
senderAddr IP address of sender.
senderPort Port of sender.

Returns:

The engine ignores the callback return value.

Called when the client gets a network packet that it cannot understand. Typically, this is used in conjunction with apps like GameSpy, etc.

Used for: ServerShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::SpecialEffectNotify( HLOCALOBJ hObj,
                                                    ILTMessage\_Read *pMessage
                                                    ) [inline, virtual]
```

Parameters:

hObj The object associated with the special effect. If NULL, then this is an "instant" special effect.
pMessage Special effect message.

Called when the server tells the client about a special effect on an object. If hObj is NULL, then this is an "instant" special effect (ie. it's not associated with an object, but instead, with a location). These messages are those that are defined by game code, and are usually (but not necessarily) sent via game code.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

```
virtual void IClientShellStub::SRand( ) [inline, virtual]
```

```
virtual void IClientShellStub::Update( ) [inline, virtual]
```

Called during every update cycle. This is the function that usually updates all client-side systems, objects, etc.

Used for: ClientShell Callbacks.

Implements [IClientShell](#).

The documentation for this class was generated from the following file:

- [iclientshell.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

IObjectPlugin Class Reference

Public Member Functions

```
virtual ~IObjectPlugin ()
virtual LTRESULT PreHook\_EditStringList (const char *szRezPath, const char *szPropName, char **aszStrings, uint32
                                     *pcStrings, const uint32 cMaxStrings, const uint32 cMaxStringLength)
virtual LTRESULT PreHook\_Dims (const char *szRezPath, const char *szPropValue, char *szModelFilenameBuf, int
                             nModelFilenameBufLen, LTVector &vDims)
virtual LTRESULT PreHook\_PropChanged (const char *szObjName, const char *szPropName, const int nPropType, const
                                   GenericProp &gpPropValue, ILTPreInterface *pInterface, const char *szModifiers)
```

Constructor & Destructor Documentation

virtual IObjectPlugin::~IObjectPlugin() [inline, virtual]

Member Function Documentation

virtual [LTRESULT](#) IObjectPlugin::PreHook_Dims(const char * *szRezPath*,
const char * *szPropValue*,
char * *szModelFilenameBuf*,
int *nModelFilenameBufLen*,
[LTVector](#) & *vDims*
) [inline, virtual]

Parameters:

<i>szRezPath</i>	Path to the rez directory for the project (input)
<i>szPropValue</i>	Value of the currently selected list item (input)
<i>szModelFilenameBuf</i>	Buffer to copy model filename into (output)
<i>nModelFilenameBufLen</i>	Max buffer length (input)
<i>vDims</i>	Dimensions to use (instead of using a model file) (output).

This hook gives the object a chance to return either the model filename to use to calculate the object's dims in DEdit, or the dims themselves (if **szModelFilename** [0] == "").

Note: This is only called on **PF_STATICLIST** properties.

If you implement this function, you should always return **LT_OK**.

Used for: Misc.

virtual [LTRESULT](#) IObjectPlugin::PreHook_EditStringList(const char * *szRezPath*,
const char * *szPropName*,
char ** *aszStrings*,
[uint32](#) * *pcStrings*,
const [uint32](#) *cMaxStrings*,
const [uint32](#) *cMaxStringLength*
) [inline, virtual]

Parameters:

<i>szRezPath</i>	Path to the rez directory for the project (input).
<i>szPropName</i>	Name of the property (input).
<i>aszStrings</i>	List of string pointers (output). Just strcpy into aszStrings[i] for string i .
<i>pcStrings</i>	How many strings were specified (output)
<i>cMaxStrings</i>	Max number of specifiable strings (input)
<i>cMaxStringLength</i>	Max length of any string in the list

This hook gives the object a chance to return an array of strings to populate a combobox edit control in DEdit. If you implement this function, you should always return **LT_OK**.

Used for: Misc.

virtual [LTRESULT](#) IObjectPlugin::PreHook_PropChanged(const char * *szObjName*,
const char * *szPropName*,
const int *nPropType*,
const [GenericProp](#) & *gpPropValue*,
[LTPreInterface](#) * *pInterface*,
const char * *szModifiers*
) [inline, virtual]

Parameters:

<i>szObjName</i>	Name of the object whos property changed (input)
<i>szPropName</i>	Name of the property that has changed (input)
<i>nPropType</i>	Type (ie. LT_PT_STRING) of the property (input)

gpPropValue The new value of the property (input)
pInterface Pointer to caller (CBaseEditObj) (input)
szModifiers String of modifiers used by the game code to alter the use of the changed property (input)

This hook gives the object a chance to check the new value to see if it is a valid value for this property.

Note: This is only called on **PF_NOTIFYCHANGE** properties.

If you implement this function, you should always return **LT_OK**.

Used for: Misc

The documentation for this class was generated from the following file:

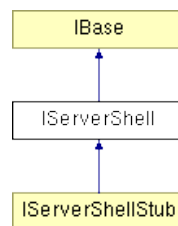
- [iobjectplugin.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
 2006, All Rights Reserved.

API Reference

IServerShell Class Reference

Inheritance diagram for IServerShell:



Public Types

```
enum { \_IServerShell\_VERSION = 4 }
```

Public Member Functions

```

ILTServer * GetServerDE () const
ILTServer * GetLTServer () const
virtual LTRESULT OnServerInitialized ()=0
    virtual void OnServerTerm ()=0
virtual LTRESULT ServerAppMessageFn (ILTMessage\_Read &msg)=0
    virtual void OnAddClient (HCLIENT hClient)=0
    virtual void OnRemoveClient (HCLIENT hClient)=0
    virtual void PreStartWorld (bool bSwitchingWorlds)=0
    virtual void PostStartWorld ()=0
virtual ILTBaseClass * OnClientEnterWorld (HCLIENT hClient)=0
    virtual void OnClientExitWorld (HCLIENT hClient)=0
    virtual void Update (LTFLOAT timeElapsed)=0
    virtual void OnMessage (HCLIENT hSender, ILTMessage\_Read *pMessage)=0
    virtual void OnObjectMessage (ILTBaseClass *pSender, ILTMessage\_Read *pMessage)=0
    virtual void OnPlaybackFinish ()=0
    virtual void CacheFiles ()=0
    virtual void SRand (unsigned int uiRand)=0
virtual LTRESULT FileLoadNotify (const char *pFilename, LTRESULT status)=0
virtual LTRESULT ProcessPacket (ILTMessage\_Read *pMsg, uint8 senderAddr[4], uint16 senderPort)=0
  
```


Detailed Description

IServerShell interface. You must implement this for a server.

Member Enumeration Documentation

anonymous enum

Enumeration values:

_IServerShell_VERSION_

Member Function Documentation

virtual void IServerShell::CacheFiles() [pure virtual]

Called after world is loaded and it's objects, their skins, and all sprites have been cached. Call [ILTServer::CacheFile\(\)](#) for each additional sprite, model, texture, and sound that you want to make sure are in memory for the level.

See also:

[ILTServer::CacheFile\(\)](#)

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

virtual [LTRESULT](#) IServerShell::FileLoadNotify(const char * *pFilename*,
[LTRESULT](#) *status*
) [pure virtual]

Parameters:

pFilename Filename.

status **LT_OK** if file loaded ok. **LT_ERROR** otherwise.

Returns:

The engine ignores the callback return value.

Called when a (model) file is done loading (from a call to ThreadLoadFile). For example, when loading a model to represent your player, [FileLoadNotify\(\)](#) is called afterwards.

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

[ILTServer](#)* IServerShell::GetLTServer() const [inline]

[ILTServer](#)* IServerShell::GetServerDE() const [inline]

virtual void IServerShell::OnAddClient([HCLIENT](#) *hClient*) [pure virtual]

Parameters:

hClient Client handle.

Called when client first connects after client has been given an ID, but before any data is sent to it.

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

[pure virtual]

virtual [ILTBaseClass](#)* IServerShell::OnClientEnterWorld([HCLIENT](#) *hClient*)

Parameters:

hClient Connecting client's handle.
pClientData Client's [StartGameRequest](#).
clientDataLen Size (in bytes) of client data.

Returns:

You MUST return the pointer to the new client instance (the C++ class, not its m_hObject).

Called after the client has actually entered the world (ie. after [OnAddClient\(\)](#)). You MUST create an object to represent the client here and return it (for PVS, etc).

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

virtual void IServerShell::OnClientExitWorld([HCLIENT](#) *hClient*) [pure virtual]

Parameters:

hClient Exiting client's handle.

Called when client leaves the world (just before the engine tells it to unload the world).

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

virtual void IServerShell::OnMessage([HCLIENT](#) *hSender*,
[ILTMessage_Read](#) * *pMessage*
) [pure virtual]

Parameters:

hSender Handle to client that sent the message.
pMessage The message.

Called when the server receives a message from a client (hSender) via SendToServer().

See also:

[ILTClient::SendToServer\(\)](#)

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

virtual void IServerShell::OnObjectMessage([ILTBaseClass](#) * *pSender*,
[ILTMessage_Read](#) * *pMessage*
) [pure virtual]

Parameters:

pSender Pointer to object that sent the message.
pMessage The message.

Called when the server receives a message from an object (pSender) via SendToServer().

See also:

[ILTServer::SendToServer\(\)](#)

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

virtual void IServerShell::OnPlaybackFinish() [pure virtual]

Called when a demo playback is done.

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

virtual void IServerShell::OnRemoveClient([HCLIENT](#) *hClient*) [pure virtual]

Parameters:

hClient Client handle.

Returns:

The engine ignores the callback return value.

Called almost immediately after [OnClientExitWorld\(\)](#) (just after client has unloaded the world).

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

virtual [LTRESULT](#) IServerShell::OnServerInitialized() [pure virtual]

Returns:

LT_OK - Everything is fine.

LT_ERROR - Fatal error occurred.

This function is called after the IServerShell implementation class has been allocated in game code, and before any other IServerShell function.

Things that used to be done in the old CreateServerShell function can now be done here.

Implemented in [IServerShellStub](#).

virtual void IServerShell::OnServerTerm() [pure virtual]

Called just before the engine unloads the servershell and shuts down.

Things that used to be done in the old DestroyServerShell function can now be done here.

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

virtual void IServerShell::PostStartWorld() [pure virtual]

Called after [CacheFiles\(\)](#), and all unused models and sounds are freed.

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

virtual void IServerShell::PreStartWorld(bool *bSwitchingWorlds*) [pure virtual]

Parameters:

bSwitchingWorlds World switching indicator. True if currently running a world. False *only* if no world is currently running (e.g. first world, or coming from a pre-game menu).

Called just after [SRand\(\)](#), before any previously loaded level objects, etc, have been unloaded.

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

virtual [LTRESULT](#) IServerShell::ProcessPacket([ILTMessage_Read](#) * *pMsg*,
 [uint8](#) *senderAddr*[4],
 [uint16](#) *senderPort*
) [pure virtual]

Parameters:

pData The packet data.
dataLen Length of the packet data.
senderAddr IP address of sender.
senderPort Port of sender.

Returns:

The engine ignores the callback return value.

Called when the server gets a network packet that it cannot understand. Typically, this is used in conjunction with apps like GameSpy, etc.

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

virtual [LTRESULT](#) IServerShell::ServerAppMessageFn([ILTMessage_Read](#) & msg) [pure virtual]

Parameters:

pMsg String message sent from the server app.
nLen Number of chars in string.

Returns:

SendToServerShell() returns the callback return value.

Receive message from the server app. Typically this is a standalone server front-end.

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

virtual void IServerShell::SRand(unsigned int *uiRand*) [pure virtual]

Parameters:

uiRand The seed that the engine passes to you (usually the system time).

First thing to be called when a world is requested, but not started loading yet. *uiRand* defaults to current time. A good use of this function would be to save the random seed for savegames, recordings, etc.

Used for: ServerShell Callbacks.

Implemented in [IServerShellStub](#).

virtual void IServerShell::Update([LTFLOAT](#) *timeElapsed*) [pure virtual]

Parameters:

timeElapsed Time since last call to [Update\(\)](#).

Called every frame tick. Typically, this function is used for synchronization, if the game requires it.

Used for: ServerShell Callbacks.

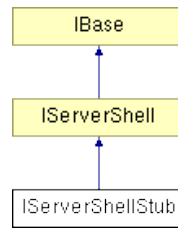
Implemented in [IServerShellStub](#).

The documentation for this class was generated from the following file:

- [iservershell.h](#)

IServerShellStub Class Reference

Inheritance diagram for IServerShellStub:



Private Member Functions

virtual [LTRESULT](#) [OnServerInitialized](#) ()
 virtual void [OnServerTerm](#) ()
 virtual [LTRESULT](#) [ServerAppMessageFn](#) ([ILTMessage_Read](#) &msg)
 virtual void [OnAddClient](#) ([HCLIENT](#) hClient)
 virtual void [OnRemoveClient](#) ([HCLIENT](#) hClient)
 virtual [ILTBaseClass](#) * [OnClientEnterWorld](#) ([HCLIENT](#) hClient)=0
 virtual void [OnClientExitWorld](#) ([HCLIENT](#) hClient)
 virtual void [PreStartWorld](#) (bool bSwitchingWorlds)
 virtual void [PostStartWorld](#) ()
 virtual void [OnMessage](#) ([HCLIENT](#) hSender, [ILTMessage_Read](#) *pMessage)
 virtual void [OnObjectMessage](#) ([ILTBaseClass](#) *pSender, [ILTMessage_Read](#) *pMessage)
 virtual void [Update](#) ([LTFLOAT](#) timeElapsed)
 virtual void [OnPlaybackFinish](#) ()
 virtual void [CacheFiles](#) ()
 virtual void [SRand](#) (unsigned int uiRand)
 virtual [LTRESULT](#) [FileLoadNotify](#) (const char *pFilename, [LTRESULT](#) status)
 virtual [LTRESULT](#) [ProcessPacket](#) ([ILTMessage_Read](#) *pMsg, [uint8](#) senderAddr[4], [uint16](#) senderPort)

Detailed Description

IServerShellStub class. This class gives empty bodies to all [IServerShell](#) functions. Derive your [IServerShell](#) class from IServerShellStub and you only need to declare functions that have non-trivial bodies.

Member Function Documentation

virtual void IServerShellStub::CacheFiles() [inline, private, virtual]

Called after world is loaded and it's objects, their skins, and all sprites have been cached. Call [ILTServer::CacheFile\(\)](#) for each additional sprite, model, texture, and sound that you want to make sure are in memory for the level.

See also:

[ILTServer::CacheFile\(\)](#)

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

virtual [LTRESULT](#) IServerShellStub::FileLoadNotify(const char * *pFilename*,
[LTRESULT](#) *status*
) [inline, private, virtual]

Parameters:

pFilename Filename.
status **LT_OK** if file loaded ok. **LT_ERROR** otherwise.

Returns:

The engine ignores the callback return value.

Called when a (model) file is done loading (from a call to ThreadLoadFile). For example, when loading a model to represent your player, [FileLoadNotify\(\)](#) is called afterwards.

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

virtual void IServerShellStub::OnAddClient([HCLIENT](#) *hClient*) [inline, private, virtual]

Parameters:

hClient Client handle.

Called when client first connects after client has been given an ID, but before any data is sent to it.

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

virtual [ILTBaseClass](#)* IServerShellStub::OnClientEnterWorld([HCLIENT](#) *hClient*) [private, pure virtual]

Parameters:

hClient Connecting client's handle.

pClientData Client's [StartGameRequest](#).

clientDataLen Size (in bytes) of client data.

Returns:

You MUST return the pointer to the new client instance (the C++ class, not its m_hObject).

Called after the client has actually entered the world (ie. after [OnAddClient\(\)](#)). You MUST create an object to represent the client here and return it (for PVS, etc).

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

virtual void IServerShellStub::OnClientExitWorld([HCLIENT](#) *hClient*) [inline, private, virtual]

Parameters:

hClient Exiting client's handle.

Called when client leaves the world (just before the engine tells it to unload the world).

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

virtual void IServerShellStub::OnMessage([HCLIENT](#) *hSender*,
[ILTMessage_Read](#) * *pMessage*
) [inline, private, virtual]

Parameters:

hSender Handle to client that sent the message.

pMessage The message.

Called when the server receives a message from a client (hSender) via SendToServer().

See also:

[ILTClient::SendToServer\(\)](#)

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

```
virtual void IServerShellStub::OnObjectMessage( ILTBaseClass * pSender,
                                              ILTMessage\_Read * pMessage
                                              ) [inline, private, virtual]
```

Parameters:

pSender Pointer to object that sent the message.
pMessage The message.

Called when the server receives a message from an object (pSender) via SendToServer().

See also:

[ILTServer::SendToServer\(\)](#)

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

```
virtual void IServerShellStub::OnPlaybackFinish( ) [inline, private, virtual]
```

Called when a demo playback is done.

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

```
virtual void IServerShellStub::OnRemoveClient( HCLIENT hClient ) [inline, private, virtual]
```

Parameters:

hClient Client handle.

Returns:

The engine ignores the callback return value.

Called almost immediately after [OnClientExitWorld\(\)](#) (just after client has unloaded the world).

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

```
virtual LTRESULT IServerShellStub::OnServerInitialized( ) [inline, private, virtual]
```

Returns:

LT_OK - Everything is fine.

LT_ERROR - Fatal error occurred.

This function is called after the [IServerShell](#) implementation class has been allocated in game code, and before any other [IServerShell](#) function.

Things that used to be done in the old CreateServerShell function can now be done here.

Implements [IServerShell](#).

```
virtual void IServerShellStub::OnServerTerm( ) [inline, private, virtual]
```

Called just before the engine unloads the servershell and shuts down.

Things that used to be done in the old DestroyServerShell function can now be done here.

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

```
virtual void IServerShellStub::PostStartWorld( ) [inline, private, virtual]
```

Called after [CacheFiles\(\)](#), and all unused models and sounds are freed.

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

```
virtual void IServerShellStub::PreStartWorld( bool bSwitchingWorlds ) [inline, private, virtual]
```

Parameters:

bSwitchingWorlds World switching indicator. True if currently running a world. False *only* if no world is currently running (e.g. first world, or coming from a pre-game menu).

Called just after [SRand\(\)](#), before any previously loaded level objects, etc, have been unloaded.

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

```
virtual LTRESULT IServerShellStub::ProcessPacket( ILTMessage\_Read * pMsg,
                                                uint8          senderAddr[4],
                                                uint16         senderPort
                                                ) [inline, private, virtual]
```

Parameters:

pData The packet data.
dataLen Length of the packet data.
senderAddr IP address of sender.
senderPort Port of sender.

Returns:

The engine ignores the callback return value.

Called when the server gets a network packet that it cannot understand. Typically, this is used in conjunction with apps like GameSpy, etc.

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

```
virtual LTRESULT IServerShellStub::ServerAppMessageFn( ILTMessage\_Read & msg ) [inline, private, virtual]
```

Parameters:

pMsg String message sent from the server app.
nLen Number of chars in string.

Returns:

[SendToServerShell\(\)](#) returns the callback return value.

Receive message from the server app. Typically this is a standalone server front-end.

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

```
virtual void IServerShellStub::SRand( unsigned int uiRand ) [inline, private, virtual]
```

Parameters:

uiRand The seed that the engine passes to you (usually the system time).

First thing to be called when a world is requested, but not started loading yet. *uiRand* defaults to current time. A good use of this function would be to save the random seed for savegames, recordings, etc.

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

```
virtual void IServerShellStub::Update( LTFLOAT timeElapsed ) [inline, private, virtual]
```


Parameters:

timeElapsed Time since last call to [Update\(\)](#).

Called every frame tick. Typically, this function is used for synchronization, if the game requires it.

Used for: ServerShell Callbacks.

Implements [IServerShell](#).

The documentation for this class was generated from the following file:

- [iservershell.h](#)

Touchdown Entertainment, Inc.
[Send feedback regarding this page.](#)
2006, All Rights Reserved.